



Foodie

Documentação Técnica — Fase P3

Rede Social de Receitas em Blazor Server

Equipa: Amanda · Cássia · Lorena

Cadeira: Programação Web

Data de Entrega: 31 de maio de 2026

Tecnologias: Blazor Server · Dapper · SQL Server · EF Core · SignalR

Este documento descreve a arquitetura, decisões técnicas, funcionalidades implementadas e as melhorias realizadas no projeto Foodie durante a fase P3.

1. Visão Geral do Projeto

O **Foodie** é uma rede social temática de receitas culinárias desenvolvida em **ASP.NET Blazor Server**. O projeto foi criado no âmbito de uma cadeira acadêmica de Programação Web, simulando um produto real com múltiplas funcionalidades sociais: publicação de receitas, sistema de seguimento entre utilizadores, grupos, favoritos, avaliações, comentários e um sistema de notificações em tempo quase real.

A fase P3 foi dedicada a corrigir problemas críticos de segurança herdados das fases anteriores, melhorar a experiência do utilizador, migrar o frontend para um design system consistente (Foodie Design System), e implementar uma feature nova — o sistema de notificações — utilizando Entity Framework Core.

Objetivos da Fase P3

Área	O que foi feito
Segurança	Corrigir SQL Injection em 8+ pontos, implementar BCrypt para passwords, padronizar sessão
Qualidade de código	Renomear Data/ → Repositories/, remover ficheiros mortos, limpar modelos duplicados
UX / Frontend	Migrar todas as páginas para o Foodie Design System (fd-* CSS classes)
Feature nova	Sistema de notificações com Entity Framework Core e polling de 30 segundos
Metodologia	3 sprints Scrum com Jira, daily standups, sprint planning e retrospectivas

2. Stack Tecnológica

Framework e Linguagem

O projeto usa **Blazor Server** (ASP.NET Core .NET 8). Blazor Server é um modelo de hosting onde os componentes Razor (.razor) correm no servidor e a UI é atualizada via WebSocket (SignalR). O browser recebe apenas HTML diferencial — não há JavaScript a ser escrito pelo programador para a lógica da app.

Porquê Blazor Server e não Blazor WASM? No contexto académico, Blazor Server facilita o acesso direto à base de dados sem expor connection strings no cliente. A latência adicional do WebSocket é aceitável numa rede local ou de desenvolvimento.

Tecnologia	Versão	Uso no projeto
ASP.NET Blazor Server	.NET 8	Framework principal — componentes .razor, routing, DI, SignalR embutido

Tecnologia	Versão	Uso no projeto
Dapper	2.x	Micro-ORM para queries SQL diretas com stored procedures e parâmetros nomeados
Entity Framework Core	8.0	Usado APENAS para o sistema de notificações (nova tabela Notifications)
SQL Server	2019+	Base de dados relacional principal. Stored procedures para operações CRUD
SignalR	embutido	Hub de chat de grupo (GroupChatHub) — implementação parcial
Blazored.SessionStorage	4.x	Gestão de sessão no browser via localStorage/sessionStorage
Syncfusion Blazor	EJ2	Componente SfAutoComplete para pesquisa de ingredientes
Bootstrap Icons	1.x	Iconografia (bi bi-*)
BCrypt.Net-Next	4.x	Hash de passwords (substituiu texto plano)
ReportLab (Python)	4.x	Geração deste documento PDF de documentação

Base de Dados — SQL Server

A base de dados usa SQL Server com stored procedures para as operações principais. As principais tabelas são: **Users** (utilizadores), **Recipes** (receitas), **Ingredients**, **Tags**, **Categories**, **Groups**, **Users_Groups**, **Favorites**, **Comments**, **Posts**, **Ratings**, **Follows**, e **Notifications** (nova tabela criada em EF Core na fase P3).

Stored procedures principais: *spRecipe_Insert*, *spRecipe_GetAll*, *spGetRecipe_GetOne*, *spRecipe_Delete*, *spIngredients_GetAll*, *spUsers_GetAll*. O Dapper chama estes procedures com parâmetros nomeados, eliminando SQL Injection.

3. Arquitetura do Projeto

Estrutura de Pastas

Pasta	Conteúdo
Pages/	Componentes Razor que mapeiam rotas (@page). Ex: Feed.razor, AddRecipe.razor
Repositories/	Acesso a dados (anteriormente Data/*Service.cs). Renomeado na P3 para refletir o padrão Repository
Models/	Classes de domínio: Recipe, User, Group, Notification, etc.
Auth/	CustomAuthenticationStateProvider para autenticação custom sem Identity
Shared/	Componentes compartilhados: NavMenu.razor, MainLayout.razor, EmptyLayout.razor
wwwroot/css/	CSS: style.css (original) + foodie.css (novo design system, fd-* classes)
Migrations/	EF Core migrations para a tabela Notifications

Padrão de Autenticação

O projeto usa um **CustomAuthenticationStateProvider** em vez do ASP.NET Identity. Após login, o email do utilizador é guardado em **session storage** com a chave **userEmail** (padronizada na P3 — anteriormente havia inconsistência entre 'email' e 'emailAddress'). O provider lê esta chave e constrói um *ClaimsPrincipal* com o email como claim.

Os componentes Blazor usam **AuthorizeView** para mostrar/esconder conteúdo baseado no estado de autenticação, sem necessitar de middleware adicional.

Padrão Repository

Cada entidade tem uma interface (ex: *IRecipeRepository*) e uma implementação concreta (*RecipeRepository*). Os repositórios são injetados via DI nos componentes Razor. Não há camada de serviço de negócio — a lógica de orquestração fica nos próprios componentes, o que é uma limitação conhecida documentada como trabalho futuro.

Sistema de Notificações — Entity Framework Core

Para demonstrar conhecimento de EF Core sem refatorar o código existente (risco elevado), foi criado um **AppDbContext** paralelo com apenas **DbSet<Notification>**. O Dapper continua a gerir todas as outras tabelas. O EF Core adiciona a tabela *Notifications* via migration sem tocar nas tabelas existentes.

```
public class Notification&ltbr/>> {  
    public int Id { get; set; }  
    public int RecipientUserId { get; set; }  
    public int? ActorUserId { get; set; }  
    public NotificationType Type { get; set; }  
    public string Message { get; set; }  
    public int? RelatedEntityId { get; set; }  
    public bool IsRead { get; set; }  
    public DateTime CreatedAt { get; set; }  
    public enum NotificationType { Follow, Comment, Rating, Favorite, GroupInvite, GroupAdmin }  
}
```

O polling de 30 segundos usa *PeriodicTimer* no NavMenu para atualizar o badge de notificações sem recarregar a página.

4. Funcionalidades Implementadas

Funcionalidade	Detalhes
Autenticação	Registo com validação, login com email+password, logout, sessão persistente, pergunta de segurança no registo
Receitas	Criar, editar, eliminar receitas com imagem, ingredientes (autocomplete Syncfusion), tags por categoria, tempo de preparação/cozinha, dificuldade
Feed	Feed personalizado (receitas de quem segues) e geral, toggle entre modos, cards com imagem, dificuldade e tempo
Favoritos	Adicionar/remover favoritos, página de favoritos do utilizador
Avaliações	Sistema de estrelas (1-5) por receita, sem avaliação dupla
Comentários	Adicionar e eliminar comentários nas páginas de receita
Grupos	Criar grupos, adicionar membros por email, promover/remover admin, sair do grupo, chat de grupo (parcial)
Perfil	Página de perfil com foto, bio, seguir/deixar de seguir, lista de receitas publicadas
Notificações	Sistema de notificações em EF Core: novo seguidor, comentário, avaliação, favorito, adicionado a grupo, promovido a admin. Badge no NavMenu, dropdown com últimas 10, polling 30s
Pesquisa	Barra de pesquisa de receitas e utilizadores, mensagem 'Sem resultados'
Categorias	Página de categoria com banner, ranking de receitas, grupos relacionados

5. Migração do Frontend — Foodie Design System

Uma das tarefas centrais da fase P3 foi a migração do frontend de um CSS heterogêneo (com estilos inline por página, classes Bootstrap, e estilos globais que 'vazavam' entre componentes) para um design system coerente e escalável.

Abordagem: ficheiro separado, sem quebrar o existente

A decisão de arquitetura foi criar um ficheiro **wwwroot/css/foodie.css** com classes prefixadas com **fd-***, ligado em `_Host.cshtml` DEPOIS do style.css original. Este approach não-destrutivo garantiu que o código existente continuou a funcionar durante toda a migração.

Porquê não modificar o style.css original? O style.css contém seletores globais (.container, a, button, etc.) que afetam toda a app. Modificá-lo causaria regressões imprevisíveis. A abordagem fd-* permite migrar página a página com controlo total.

Estrutura do Design System (foodie.css)

Categoria	Classes / Tokens
Variáveis CSS	--orange, --orange-2, --surface, --line, --ink, --ink-2, --muted, --r-sm, --r-lg, --font-display
Layout	fd-page-wrap, fd-container, fd-grid-2, fd-grid-3, fd-row, fd-row-3col, fd-flex, fd-flex-between
Botões	fd-btn, fd-btn-primary (laranja), fd-btn-outline, fd-btn-success, fd-icon-btn
Cards	fd-card, fd-card-header, fd-card-body
Formulários	fd-form-group, fd-form-label, fd-form-control, fd-form-select, fd-input-row
Imagens placeholder	fd-img, fd-img--a/b/c/d/e (5 cores de gradiente alternadas por ID)
Avatares	fd-avatar--sm, fd-avatar--xs, fd-avatar--xl, fd-avatar--a a --e
Badges	fd-badge, fd-badge-success, fd-badge-warning, fd-badge-danger, fd-badge-admin, fd-badge-member
Chat	fd-chat, fd-bubble, fd-bubble--me (laranja), fd-bubble__body, fd-chat-input
Modais	fd-modal-overlay, fd-modal, fd-modal-footer
Landing/Auth	fd-empty-layout, fd-empty-header, fd-landing, fd-auth-overlay, fd-auth-panel
Notificações	fd-notif-dot, fd-notif-dropdown, fd-notif-item, fd-notif-item--unread
Feed	fd-recipe-card, fd-feed-toggle, fd-create-card, fd-tag-pill, fd-thumb
Outros	fd-banner, fd-banner__overlay, fd-group-item, fd-group-icon, fd-pw-bar, fd-pw-checks

Páginas Migradas

Página	O que foi migrado
Index.razor	Landing page com SVG ilustrativo, modais de login/registo em overlay, EmptyLayout
Login.razor	Modal overlay fd-auth-overlay, validação de erro, callback para abrir registo
Register.razor	Modal com barra de força de password (fd-pw-bar), checklist animada, scroll interno
Feed.razor	Layout 3 colunas, fd-recipe-card com imagem, badges de dificuldade, toggle feed
ShowRecipe.razor	Grid de receitas fd-grid-2, placeholder de cor por ID, modal de confirmação de eliminação
AddRecipe.razor	Formulário 2 colunas, SfAutoComplete ingredientes, details/summary para tags, validação
EditRecipe.razor	Preservação de foto existente, details/summary para categorias, botões Confirmar/Cancelar
GroupDetail.razor	Layout 2 colunas, chat com fd-bubble/fd-bubble--me, formulário de mensagem
GroupPage.razor	Lista expansível de grupos, modais de adicionar membro e promover admin
CategoryPage.razor	Banner com overlay, grid de ranking, grupos relacionados, sidebar de ícones
NavMenu.razor	Dropdown de perfil com estado Blazor, sino de notificações com badge, fecho automático

Problema: CSS do style.css a sobrescrever fd-btn-primary

O style.css original tem uma regra global a `{ color: #8f8f8f }` que sobrescrevia a cor do texto nos botões do tipo `<a>`. A solução foi adicionar `color: #fff !important` ao `fd-btn-primary` e `.fd-page-wrap` a `{ color: var(--orange-2) !important }` para links normais.

Este é um exemplo típico de coexistência de dois sistemas CSS: o `!important` foi necessário devido à especificidade maior do seletor global herdado.

6. Bugs Identificados e Corrigidos na P3

P0 — Críticos (Segurança)

ID	Bug	Responsável	Solução
BUG-01	SQL Injection em 8+ pontos	Cássia	Trocar interpolação de string por parâmetros Dapper (@Email, @Password, etc.) em todos os services
BUG-02	Passwords em texto plano	Cássia	Implementar BCrypt.Net-Next: HashPassword no registo, Verify no login. Migration de dados existentes
BUG-03	Chave de sessão inconsistente ('email' vs 'emailAddress')	Amanda	Find & Replace em todo o projeto para userEmail. Resolve grupos que não carregavam após registo
BUG-05	Logout não funciona (href=" recarregava página)	Amanda	Implementar logout real: limpar sessionStorage, chamar MarkUserAsAuthenticated(null), redirecionar para /

P1 — Bugs Visíveis ao Utilizador

ID	Bug	Responsável	Solução
BUG-04	Login não aceita clique após registo	Cássia	Corrigir estado de autenticação pós-registo (auto-login após criar conta)
BUG-06	Recomendações faziam logout ao clicar	Cássia	Corrigir link de navegação no componente de recomendações
BUG-07	Estrelas não guardavam após avaliar	Cássia	Corrigir chamada ao repositório de ratings, bloquear avaliação dupla por utilizador
BUG-08	Favoritos não atualizavam na UI	Cássia	Chamar StateHasChanged() após toggle de favorito
BUG-09	Receitas em 'Minha Página' não clicáveis	Amanda	Adicionar link correto /recipepage/@rec.Id_Recipe nos cards
BUG-10	Não permitia trocar foto de perfil	Amanda	Corrigir query UPDATE no UserRepository para incluir campo UserPhoto
BUG-11	Editar receita: foto desaparecia	Amanda	Preservar imagem existente se utilizador não carregar nova
BUG-16	favoriteCount igual em todos os cards do Feed	Cássia	favoriteCount era variável única (não por receita). Corrigir para dicionário por Id_Recipe
BUG-17	Feed com duplo carregamento	Cássia	Remover duplicação OnInitializedAsync + OnParametersSetAsync

ID	Bug	Responsável	Solução
BUG-DE L	Delete de receita falha (FK constraint)	Cássia	Stored procedure spRecipe_Delete corrigida para apagar dependências (Tags, Ingredients, Favorites, Comments) antes da receita

Bugs de Qualidade Técnica (P2)

N+1 queries no Feed: O loop de carregamento de receitas fazia uma query por receita para buscar tags e favorite count. Corrigido com JOIN no SQL que retorna tudo de uma vez.

N+1 em comentários: O loop de comentários fazia uma query por comentário para buscar dados do utilizador. Corrigido com JOIN em LoadRecipeComments para incluir Username e UserPhoto.

CSS inline a vazar entre componentes: Páginas com <style> inline contaminavam o CSS global. Migrado para ficheiros .razor.css isolados por componente.

7. Metodologia — Scrum com Jira

O projeto seguiu metodologia **Scrum** com 3 sprints de 1 semana cada, geridos no **Jira**. Cada sprint teve planning na segunda-feira, daily standups de 15 minutos, e review/retro na sexta.

Sprint	Datas	Foco	Tarefas principais
Sprint 1	11–15 maio 2026	Segurança e bugs críticos	SQL Injection, BCrypt, sessão userEmail, logout, validação de registo, setup EF Core
Sprint 2	18–22 maio 2026	Bugs UX + início de notificações	Avaliações, favoritos, foto de perfil, editar receita, INotificationService, NotificationRepository
Sprint 3	25–29 maio 2026	Notificações + polish + QA	UI de notificações (sino, dropdown, polling), melhorias UX, tradução PT, testes manuais, documentação

Distribuição de Trabalho

Pessoa	Área	Estimativa	Responsabilidades
Amanda	Front-end + UX	~26 SP / ~13 dias	Migração de páginas para Foodie Design System, foodie.css, UX improvements, tradução PT, QA checklist, documentação
Cássia	Backend + Segurança + Bugs	~26 SP / ~13 dias	SQL Injection, BCrypt, sessão, bugs UX (favoritos, avaliações, foto), limpeza técnica (Repositories/)
Lorena	Notificações + Estrutura	~27 SP / ~13.5 dias	EF Core setup, modelo Notification, INotificationService, UI sino, polling, consolidar Startup.cs

Backlog — 7 Epics no Jira

Epic	Nome	Scope	Assignee	Sprint
EPIC-1	Segurança	SQL Injection, BCrypt, validação de registo	Cássia	Sprint 1
EPIC-2	Bugs Críticos de Fluxo	Sessão, logout, login pós-registo, recomendações	Amanda + Cássia	Sprint 1
EPIC-3	Bugs UX	Avaliações, favoritos, foto, editar receita, favoriteCount	Amanda + Cássia	Sprint 1-2
EPIC-4	Melhorias UX	Foto genérica, botões, tempo numérico, rendimento, dificuldade	Amanda + Lorena	Sprint 2-3

Epic	Nome	Scope	Assignee	Sprint
EPIC-5	Notificações (EF Core)	Setup EF, modelo, service, UI sino, polling	Lorena	Sprint 1-3
EPIC-6	Limpeza Técnica	Ficheiros mortos, renomear Repositories/, modelos duplicados	Cássia + Lorena	Sprint 1+3
EPIC-7	QA e Documentação	Testes manuais, PDF documentação, README	Amanda	Sprint 3

Convenção de Story Points: 1 SP = ~0.5 dia · 2 SP = ~1 dia · 3 SP = ~1.5 dias · 5 SP = ~2.5 dias.
Total estimado: ~42 person-days para 3 pessoas em 15 dias úteis.

8. Limitações Conhecidas e Trabalho Futuro

As seguintes limitações são conhecidas e documentadas intencionalmente. São mencionadas na apresentação como 'trabalho futuro' em vez de escondidas.

Limitação	Detalhes
Sem paginação	O feed carrega todas as receitas da BD. Para produção seria necessário paginação ou scroll infinito.
Imagens em base64 inline	Imagens são convertidas para base64 e embutidas no HTML. Para escalabilidade, deviam ser servidas por endpoint GET <code>/api/recipe/{id}/image</code> com cache de browser.
Chat de grupo parcial	O SignalR Hub (GroupChatHub) está implementado mas não persiste mensagens na BD. A feature foi mantida na estrutura mas não exposta na UI final.
Sem lógica de negócio separada	A orquestração de lógica fica nos componentes Razor. Deveria existir uma camada Services/ com AuthService, FeedService, etc.
Naming inconsistente na BD	Convenções mistas (Id_User vs IdUser, Id_Recipe vs IdRecipe) herdadas da fase P1. Mudar implicaria migration de dados e atualização de queries.
CategoryPage com conteúdo estático	Alguns dados da página de categoria são hardcoded ('Cozinha Italiana'). Falta ligação dinâmica às categorias da BD.
Sem testes automatizados	Apenas testes manuais. Para produção: xUnit + bUnit para testes de componentes Blazor.
Login Google / OAuth	Fora de scope desta entrega. Requer projeto no Google Developer Console e pacote Microsoft.AspNetCore.Authentication.Google.

9. Perguntas e Respostas — Preparação para a Apresentação

Esta seção prepara a equipe para as perguntas mais prováveis de um professor avaliador. As respostas são diretas e técnicas.

P: O que é Blazor Server e por que o usaram em vez de React ou Angular?

R: Blazor Server é um framework da Microsoft onde os componentes correm no servidor e a UI é atualizada via WebSocket (SignalR). Escolhemos Blazor Server porque: (1) permite usar C# em vez de JavaScript, mantendo consistência com o backend; (2) o acesso direto à base de dados é mais simples sem expor APIs públicas; (3) era o requisito da cadeira. A desvantagem é a latência do WebSocket e a escalabilidade limitada.

P: O que é Dapper e como difere do Entity Framework?

R: Dapper é um micro-ORM que executa SQL diretamente e mapeia os resultados para objetos C#. É mais rápido que EF mas requer escrever SQL manualmente. EF Core é um ORM completo com migrations, LINQ, change tracking e geração automática de SQL. Usamos Dapper para o código existente (onde já havia stored procedures) e EF Core apenas para a feature nova de notificações, demonstrando conhecimento de ambos sem refatorar o código existente.

P: Porquê usar stored procedures em vez de SQL inline?

R: Stored procedures oferecem: (1) separação entre lógica de acesso a dados e aplicação; (2) potencial de performance (query plan caching no SQL Server); (3) menor superfície de ataque se os parâmetros forem sempre passados corretamente. No nosso caso, as stored procedures foram herdadas de fases anteriores. O problema original era que algumas queries no código (não em SPs) usavam interpolação de string — corrigimos para parâmetros Dapper em todos os casos.

P: O que é SQL Injection e como corrigiram?

R: SQL Injection ocorre quando input do utilizador é concatenado diretamente numa query SQL. Por exemplo: `SELECT * FROM Users WHERE Email = '{Email}'` — se Email for `' OR '1'='1'`, a query retorna todos os utilizadores. Corrigimos substituindo interpolação por parâmetros nomeados do Dapper: `SELECT * FROM Users WHERE Email = @Email`, passando `new { Email }` como objeto. O Dapper sanitiza automaticamente os valores.

P: Como funciona a autenticação sem ASP.NET Identity?

R: Implementámos um CustomAuthenticationStateProvider. Após login, guardamos o email em sessionStorage com a chave 'userEmail'. O provider lê esta chave e constrói um ClaimsPrincipal com o email como claim Name. Os componentes usam para mostrar/esconder conteúdo. O logout limpa o sessionStorage e chama MarkUserAsAuthenticated com null. É uma solução mais simples que o Identity, adequada para o contexto académico.

P: Porquê BCrypt para passwords e não SHA256 ou MD5?

R: MD5 e SHA256 são funções de hash rápidas — um atacante pode testar biliões de combinações por segundo com hardware moderno (rainbow tables, brute force). BCrypt é desenhado para ser lento: tem um fator de custo configurável que aumenta o tempo de hash. Além disso, BCrypt inclui um salt aleatório automaticamente em cada hash, tornando ataques de dicionário ineficazes. BCrypt.Net-Next é a biblioteca padrão para .NET.

P: O que é o Foodie Design System e como foi criado?

R: O Foodie Design System é um conjunto de classes CSS com prefixo fd-* (ex: fd-btn, fd-card, fd-grid-2) definidas num ficheiro separado foodie.css. Foi criado para substituir o CSS heterogéneo existente (estilos inline, classes Bootstrap misturadas, estilos globais que vazavam entre componentes) por um sistema consistente com tokens de design (variáveis CSS --orange, --surface, etc.). A abordagem não-destrutiva — adicionar o ficheiro sem modificar o style.css original — permitiu migrar página a página com segurança.

P: Como funciona o sistema de notificações em Entity Framework Core?

R: Criámos um ApplicationDbContext paralelo com apenas DbSet. O EF Core gere apenas a tabela Notifications via migration, sem tocar nas tabelas existentes do Dapper. A INotificationRepository expõe CreateAsync, GetUnreadCountAsync, GetRecentAsync e MarkAllAsReadAsync. O NavMenu usa PeriodicTimer para fazer polling a cada 30 segundos e atualizar o badge de notificações. Cada ação relevante (seguir, comentar, avaliar, favoritar, adicionar a grupo) chama NotificationRepository.CreateAsync.

P: O que é SignalR e como usaram no projeto?

R: SignalR é uma biblioteca que facilita comunicação bidirecional em tempo real entre servidor e cliente via WebSocket (com fallback para long polling). No Blazor Server, o SignalR já é usado internamente para a sincronização de componentes. No nosso projeto, implementámos adicionalmente um GroupChatHub para o chat de grupos. O hub não chegou a ser completamente integrado com UI e persistência de mensagens na BD — ficou como limitação conhecida e trabalho futuro.

P: O que é o padrão Repository e como o aplicaram?

R: O padrão Repository abstrai o acesso a dados atrás de uma interface. Por exemplo, IRecipeRepository define GetRecipesByUserId, AddRecipe, DeleteRecipe, etc. A implementação concreta RecipeRepository usa Dapper para executar as queries. Os componentes Razor injetam a interface via DI — nunca a implementação concreta. Isto facilita testes (mock da interface) e troca de tecnologia de dados sem alterar os componentes.

P: Como gerem o estado entre componentes no Blazor?

R: Usamos múltiplas estratégias: (1) Blazored.SessionStorage para estado persistente (email do utilizador logado); (2) parâmetros e EventCallbacks entre componentes pai/filho (ex: Login.razor chama OnOpenRegister para abrir o modal de registo); (3) variáveis de instância no @code do componente para estado local (ex: showDeleteModal, menuOpen, notifOpen); (4) StateHasChanged() quando o estado muda fora do ciclo de vida normal do Blazor (ex: após polling de notificações).

P: Como trataram a questão de imagens nas receitas?

R: As imagens são guardadas como byte[] na coluna Image da tabela Recipes no SQL Server. Nos componentes, convertemos para base64 inline: `data:image/jpeg;base64,{Convert.ToBase64String(rec.Image)}`. A limitação é que para listagens com muitas receitas, o payload pode ser pesado (50MB+ para 100 receitas). A solução ideal — não implementada por falta de tempo — seria um endpoint `GET /api/recipe/{id}/image` que serviria a imagem com cache de browser.

P: O que mudaram na stored procedure `spRecipe_Delete` e porquê?

R: A stored procedure original não apagava registos dependentes antes de apagar a receita, causando erro de foreign key constraint. A colega Cássia corrigiu a SP diretamente no SQL Server para apagar primeiro os registos em `Recipes_Tags`, `Recipes_Ingredients`, `Favorites`, `Comments` e `Ratings` que referenciam a receita, e só depois apagar a linha em `Recipes`. A alternativa teria sido `ON DELETE CASCADE` nas foreign keys, mas implicava alterar o schema da BD com risco de impacto noutras operações.

P: Como funcionam os 'details/summary' na adição de receitas?

R: Em vez de usar o accordion do Bootstrap (que requer JavaScript), usámos o elemento HTML nativo `/`. Ao clicar no `,` o browser expande/colapsa o `<div>` sem nenhum JavaScript. Isto simplifica o código, elimina dependência de Bootstrap JS, e funciona em todos os browsers modernos. Usámos este approach nos formulários de tags e categorias em `AddRecipe.razor` e `EditRecipe.razor`.

P: O que é o `CustomAuthenticationStateProvider` e como se integra com `AuthorizeView`?

R: `CustomAuthenticationStateProvider` herda de `AuthenticationStateProvider` e é registado no DI como singleton. O método `GetAuthenticationStateAsync` lê o email do `sessionStorage` e constrói um `ClaimsPrincipal`: se há email, o utilizador está autenticado; se não, é anónimo. O método `MarkUserAsAuthenticated(email)` atualiza este estado e dispara `NotifyAuthenticationStateChanged`, o que faz o Blazor re-renderizar todos os componentes que usam `AuthorizeView`. Este provider é injetado em `Login.razor` e `Register.razor`.

P: Porquê não migraram para Entity Framework Core completamente?

R: O risco/benefício era negativo neste prazo: (1) o projeto tem stored procedures em SQL Server que ficariam órfãs numa migração para EF; (2) reescrever ~50 ficheiros de acesso a dados em 15 dias úteis com 3 pessoas era inviável; (3) o Dapper não é o problema — o problema era como estava a ser usado (interpolação vs parâmetros). Decidimos usar EF Core apenas para a feature nova de notificações, demonstrando que sabemos usar EF sem partir o que existe.

P: Como trataram conflitos de merge no Git com 3 pessoas a trabalhar em paralelo?

R: Adoptámos branches por epic/feature (ex: feature/SEC-1-sql-injection, feature/notifications-ef). Cada pessoa trabalhou em ficheiros diferentes sempre que possível: Amanda nos ficheiros .razor e foodie.css, Cássia nos Repositories e modelos, Lorena em ficheiros novos (Notifications). Fizemos rebase frequente para evitar conflitos grandes. O maior risco identificado era tocar em ficheiros comuns como NavMenu.razor e Feed.razor.

P: O que é um PeriodicTimer e como o usam nas notificações?

R: PeriodicTimer é uma classe .NET 6+ que permite aguardar intervalos de tempo de forma eficiente sem bloquear threads. No NavMenu, criámos um PeriodicTimer com intervalo de 30 segundos num loop assíncrono: `while (await timer.WaitForNextTickAsync()) { unreadCount = await NotificationRepo.GetUnreadCountAsync(userId); StateHasChanged(); }`. O timer é cancelado quando o componente é eliminado (IAsyncDisposable). É mais eficiente que `Task.Delay` em loop porque não aloca uma Task por tick.

P: Como foi a gestão do projeto com Scrum?

R: Usámos Jira com 7 epics e ~42 stories importadas via CSV. Cada sprint teve: planning de 30 min na segunda para atribuir stories e estimar em SP; daily standup de 15 min para sincronizar bloqueios; review/retro na sexta para demonstrar trabalho feito e identificar melhorias. A distribuição foi equilibrada: ~26-27 SP por pessoa em 15 dias úteis. O maior risco materializado foi a EF Core demorar mais que o esperado no Sprint 1.

10. Definition of Done (DoD)

Uma story só é considerada 'Done' quando satisfaz todos os seguintes critérios:

- Código implementado e **compila sem warnings novos**
- Testado manualmente pela pessoa que implementou
- Funciona em ambos os fluxos (registo novo e login existente)
- Não introduziu SQL Injection nem credenciais hardcoded no código
- Sem Console.WriteLine de debug deixado para trás
- Commit com mensagem clara referenciando o ticket Jira (ex: feat(SEC-1): corrigir sql injection em LoginRepository)
- PR revisto por pelo menos outra pessoa antes de merge para dev
- Sem regressões nas funcionalidades existentes (teste de regressão manual)

11. Glossário

Termo	Definição
Blazor Server	Framework web da Microsoft onde componentes Razor correm no servidor e a UI sincroniza via SignalR/WebSocket
Dapper	Micro-ORM para .NET: executa SQL e mapeia resultados para objetos C# com overhead mínimo
EF Core	Entity Framework Core — ORM completo da Microsoft com migrations, LINQ e gestão de relações
SignalR	Biblioteca para comunicação em tempo real (WebSocket) entre servidor e cliente
Repository Pattern	Padrão de design que abstrai o acesso a dados atrás de uma interface, desacoplando a lógica de negócio da persistência
BCrypt	Algoritmo de hash de passwords com salt incorporado e fator de custo configurável, resistente a brute force
SQL Injection	Vulnerabilidade onde input do utilizador é concatenado em queries SQL, permitindo manipulação da BD
Story Points (SP)	Unidade relativa de estimativa de esforço em Scrum (1 SP $\approx$ 0.5 dia neste projeto)
fd-* classes	Classes CSS prefixadas com 'fd-' pertencentes ao Foodie Design System (foodie.css)
sessionStorage	API do browser para guardar dados por sessão (desaparecem ao fechar o tab)

Termo	Definição
PeriodicTimer	Classe .NET 6+ para executar código em intervalos regulares de forma eficiente e assíncrona
AuthorizeView	Componente Blazor que mostra/esconde conteúdo baseado no estado de autenticação
ClaimsPrincipal	Objeto .NET que representa o utilizador autenticado com os seus claims (ex: email, role)
N+1 Problem	Anti-padrão de performance onde se fazem N queries adicionais para cada item de uma lista de N itens
Stored Procedure	Bloco de SQL pré-compilado guardado no servidor de BD, chamado por nome com parâmetros

Foodie · Documentação P3
Amanda · Cássia · Lorena — Maio 2026
Blazor Server · Dapper · EF Core · SQL Server · SignalR

Documento gerado automaticamente a partir da análise técnica do projeto.